CBIT HACKTOBERFEST '26 · TEAM 48 · CONVERGE

ShadowGuard AI

Detecting AI Shadow IT and agentic data exfiltration — without decrypting a single byte

---

TEAM

Chetan Varma Bethalam
Team lead · Frontend

Mohammed Muzammil
Backend & integration

Sai Krishna
Demo data & live collector

Anurag Kanukuntla
Documentation & QA

Nimmala Karthik
Featherless & pitch

Problem track: AI Shadow IT & Agentic Data Exfiltration Scanner · CBIT Hyderabad

01 · THE PROBLEM

AI is leaking company data — and nobody sees it

People and autonomous agents send secrets to AI tools the company never approved

1. **Shadow AI**
   ChatGPT, Claude, local Ollama models, Hugging Face — used without approval. Security has no inventory.

2. **Secrets inside prompts**
   API keys, .env files and customer data (Aadhaar, cards) get pasted into prompts to "debug faster".

3. **Autonomous agents**
   Agents read files, run shell commands and call LLMs on their own — no human ever pastes anything.

THE PROBLEM STATEMENT ASKS US TO

1. Inspect local connections or uploaded request logs
2. Identify AI providers and local LLMs (e.g. port 11434)
3. Detect secrets and PII with regex + entropy
4. Flag risky agent actions (shell, file reads)
5. Show a local, privacy-first risk dashboard

Constraint: no SSL decryption, no heavy enterprise agent

01 · THE GAP

Why existing tools miss it

They were built for humans and websites — not for local LLMs and autonomous agents

Existing tool	What it sees	What it misses
Firewall / proxy logs	Domains and IPs	Local LLMs and which app sent the request
Enterprise DLP / CASB	Decrypted content	Needs TLS interception on every device; costly
AI gateways / guardrails	Only routed traffic	Shadow tools that bypass the gateway
Secret scanners (Git)	Code in repos	Live prompts and agent actions

⚠ An agent that reads **.env**, runs a shell command and sends the key looks like **one ordinary HTTPS request** to every network tool. The file read and the shell command happen on the laptop.

02 · OUR SOLUTION

ShadowGuard AI

A lightweight local engine that discovers shadow AI, catches leaks and explains every alert

Zero-decryption Shadow AI inventory

Process → AI provider → Approved or SHADOW, built from connection metadata. No TLS interception.

Agentic exfiltration chain detection

File read → shell command → credential sent within 60 s becomes one critical incident, shown as a timeline.

Privacy-preserving AI triage

The Featherless LLM explains incidents and suggests fixes from redacted fields only — never a payload or raw secret.

Explainable 0–10 risk score · **Luhn · Verhoeff · JWT validators** · **India-aware PII: Aadhaar, PAN, +91** · **Keyed HMAC fingerprints** · **Runs locally on 127.0.0.1**

02 · WHAT WE CAN SEE

Two tiers of visibility, zero decryption

TIER 1 · ALL TRAFFIC, ALWAYS ON

Connection metadata

The collector reads the OS connection table every 3 s with **psutil** — like **netstat**.

```
chrome.exe → api.anthropic.com:443 → SHADOW
```

Sees: program, PID, AI provider, time, approved or shadow.
Not: the content of encrypted HTTPS.

TIER 2 · WHERE PLAINTEXT EXISTS

Full payload inspection

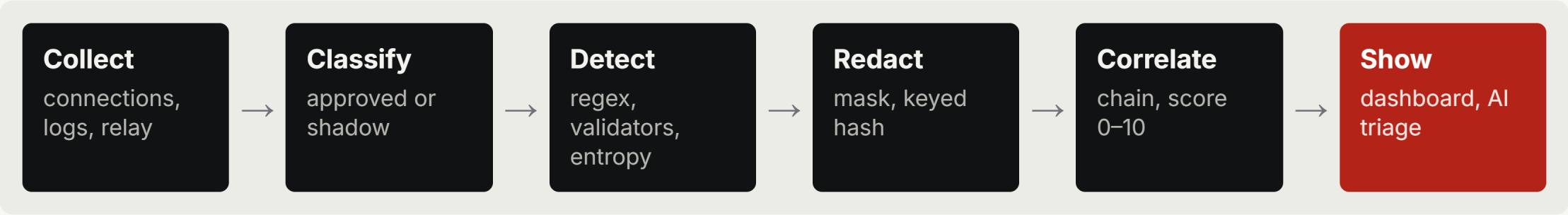
- **Request logs** from AI gateways, SDKs, agents
- **Local LLMs:** Ollama on :11434 (plain HTTP)
- **Live Test Console** for hands-on tests

Sees: the full prompt → secrets, PII and agent tool calls.

We never decrypt TLS. Reading data where the organisation already holds it in plaintext is legitimate; intercepting someone's encrypted traffic is not.

03 · ARCHITECTURE

From raw event to explained incident



BACKEND

Python, FastAPI, Pydantic, SQLite, psutil

Fast to build, one local file DB, reads the OS connection table

DETECTION

Regex, Luhn, Verhoeff, JWT check, entropy

Deterministic and explainable — no model training needed

FRONTEND

React, Vite, Tailwind, Recharts

Live dashboard that refreshes every 3 seconds

AI LAYER

Featherless · Qwen2.5-7B-Instruct

Explains only; offline rules if the LLM is unavailable

04 · INSIDE THE ENGINE

Following one leaked AWS key

RAW REQUEST · PYTHON.EXE → LOCAL OLLAMA

```
POST localhost:11434/api/chat
process: python.exe · pid 4412
{"model": "llama3",
"messages": [{"role": "user",
"content": "deploy fails:
AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
region ap-south-1"}]}
```

1. **Classify the destination**
   port 11434 → Ollama → not approved → **SHADOW** · +2
2. **Match the pattern**
   regex **AKIA + 16 caps/digits** → Credential · AWS_Access_Key · +4
3. **Redact in memory**
   evidence **AKIA**********MPLE** + keyed fingerprint (HMAC); raw prompt dropped
4. **Score and raise an incident**
   High · 6/10 — “python.exe sent an AWS key to unapproved provider Ollama.”

04 · INSIDE THE ENGINE

Pattern + proof: how we avoid false alarms

Sample input	Regex candidate → validator	Result
3043 3218 1964	Aadhaar shape → Verhoeff checksum passes	Flagged · PII Aadhaar
2234 5678 9012	Aadhaar shape → Verhoeff checksum fails	Ignored
4111 1111 1111 1111	Card shape → Luhn checksum passes	Flagged · PII card
4111 1111 1111 1112	Card shape → Luhn checksum fails	Ignored
eyJhbGciOiJIUzI1NiJ9....	JWT shape → header decodes to {"alg":"HS256"}	Flagged · JWT
550e8400-e29b-41d4-...	Random-looking → UUID excluded	Ignored
...	Random-looking → image data excluded	Ignored

Also detected: AWS secret keys, OpenAI, Anthropic, GitHub and Hugging Face tokens, private keys, email, +91 phone, PAN, high-entropy strings. **Bait file 06** (UUIDs, git hashes, images, invalid numbers): **0 findings.**

05 · AGENTIC EXFILTRATION

Catching an AI agent that steals secrets

autogpt is asked to “fix the deployment script”. No human pastes anything.

T+0s	**FILE READ** `/home/dev/project/.env`	+3
T+3s	**SHELL COMMAND** `cat ~/.aws/credentials`	+3

T+6s	**AI REQUEST → api.openai.com**
	`AKIA… key in message`
	`tool_call run_shell(cat ~/.aws/…)`
		+4 +3

Events arrive one at a time; ShadowGuard re-scores the whole session on each one.

SCORE, LIVE

3 → 6 → 10

Sensitive file read · +3

Agent shell execution · +3

Credential found · +4

Chain: read → key in 60 s · +3

13 points, capped at **10** →

CRITICAL · 10/10

05 · RISK SCORING

Every point is explainable

Factor (each counted once per incident)	Points
Credential found (API key, token, private key)	+4
Agent shell execution	+3
Sensitive file read (.env, .ssh, .aws)	+3
Chain: sensitive read → credential within 60 s	+3
PII: 1–2 items / 3 or more	+2 / +3
Unsanctioned (shadow) AI provider	+2
High-entropy secret-like string	+2

Why points, not ML? Reproducible, auditable, no training data — an analyst sees exactly why.

SEVERITY (SCORE CAPPED AT 10)

Critical · 8–10

High · 5–7

Medium · 3–4

Below 3 · logged, no incident

EXAMPLES

Key → OpenAI (approved) · **4 Medium**

Key → Ollama (shadow) · **6 High**

Key + 3 PII → Ollama · **9 Critical**

Agent chain · **13 → 10 Critical**

06 · PRIVACY BY DESIGN

A security tool must not become the next leak

NEVER STORED

Raw payload

```
AKIAIOSFODNN7EXAMPLE
3043 3218 1964
“deploy fails…”
```

Scanned in memory, then dropped. The database has no payload column.

STORED LOCALLY

Redacted metadata

```
AKIA**********MPLE
XXXX XXXX 1964
fp 3370014c7b89
```

Keyed HMAC fingerprint spots repeat leaks; useless without the local key.

SENT TO FEATHERLESS

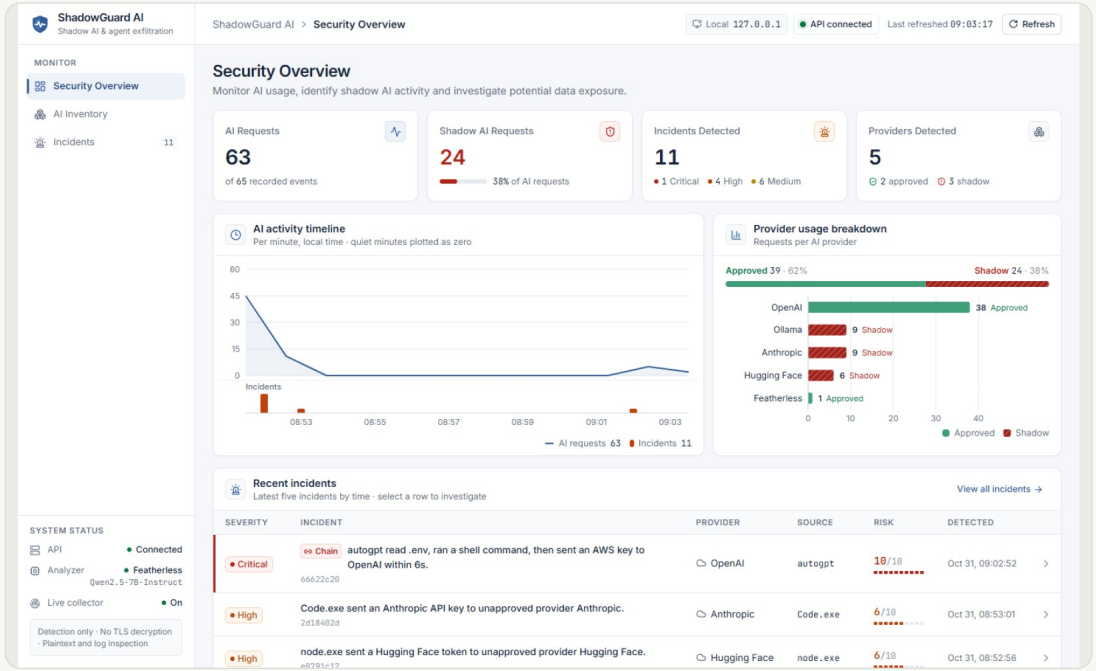
Redacted fields

```
provider · severity
score breakdown
redacted evidence
```

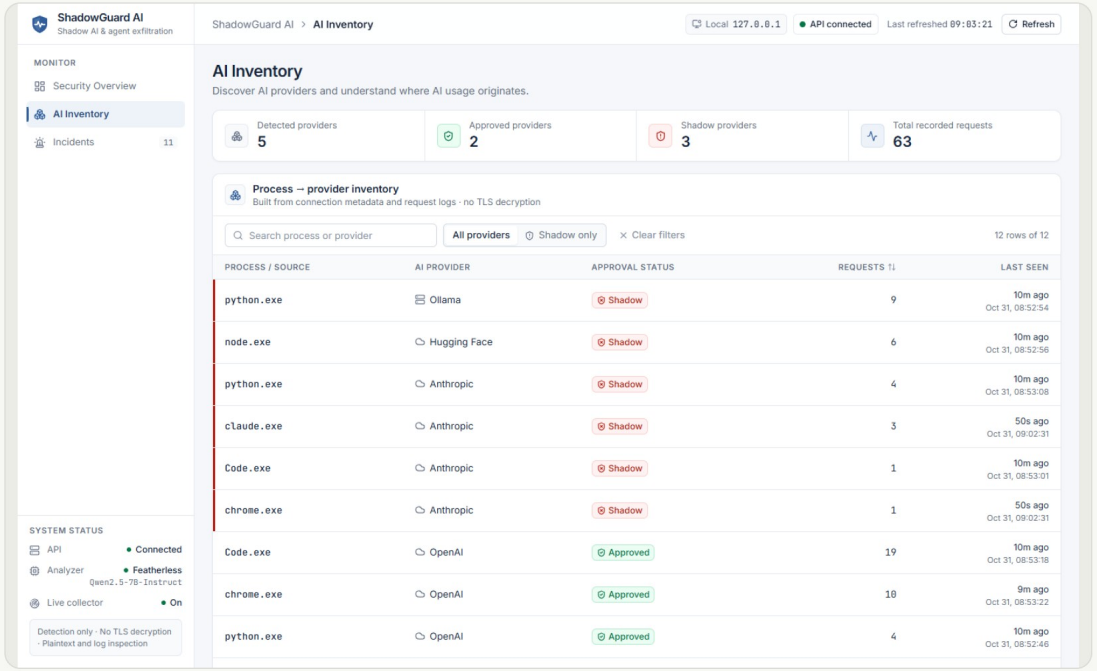
No payloads, no raw secrets, no fingerprints. Offline rules if the LLM is down.

Runs on **127.0.0.1** · the only outbound call is the optional AI analysis, on an admin’s click

Security Overview and AI Inventory

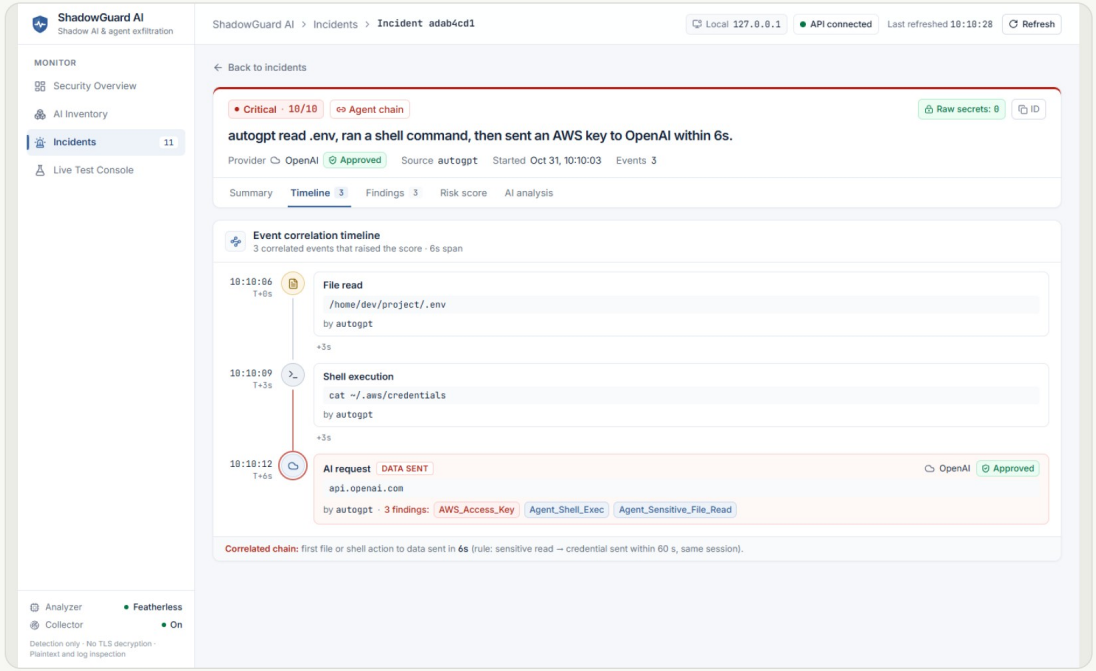


Overview: AI requests, shadow AI share, incidents by severity, approved vs shadow traffic per provider.

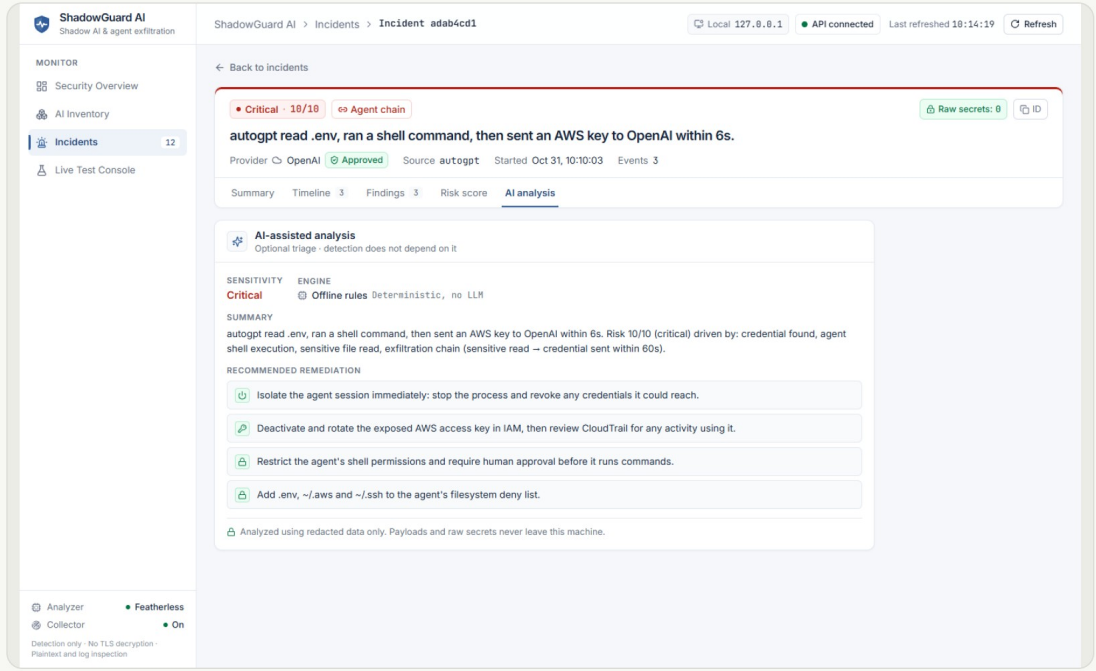


AI Inventory: which program uses which provider. Rows like claude.exe → Anthropic are real, from the live collector.

Investigating an incident



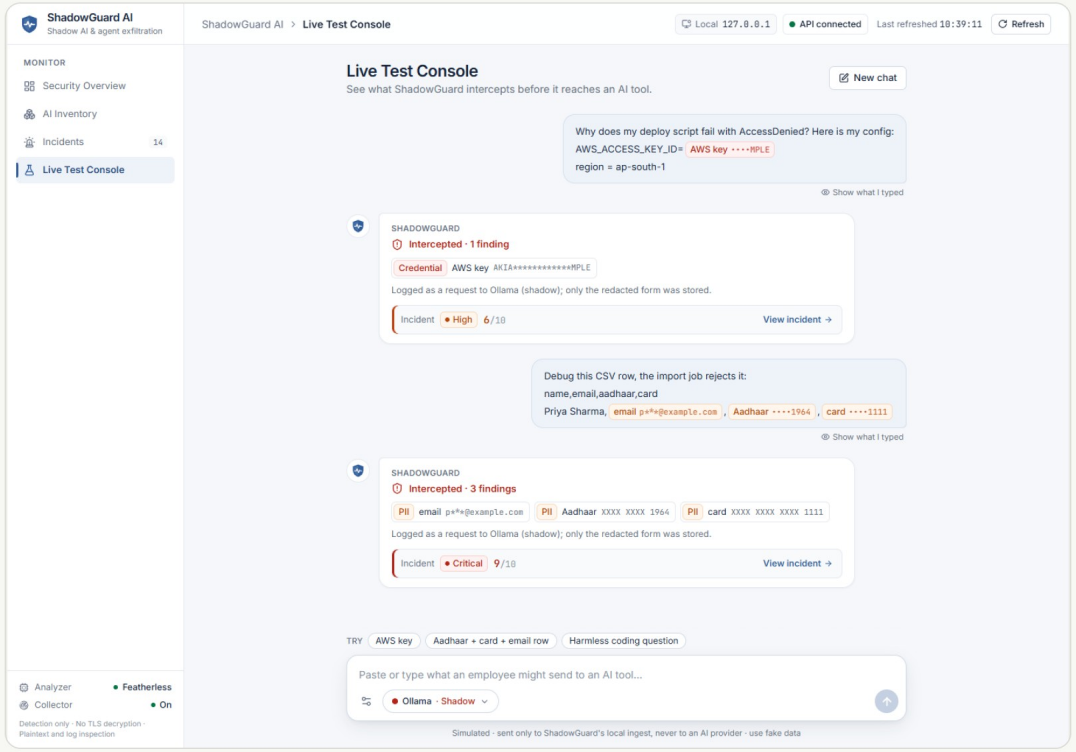
Timeline: .env read → shell command → AWS key sent to OpenAI, 3 s apart — one critical incident.



AI analysis: summary and remediation checklist, generated from redacted data only.

07 · LIVE PROOF

Not hardcoded: test it with your own data



LIVE TEST CONSOLE

Type it, watch it get caught

A judge types any fake key or customer row; ShadowGuard masks it, scores it and opens the incident.

LOCAL LLM GUARD

Real local LLM traffic

An opt-in relay in front of Ollama reads real prompts — plain HTTP, observe-only.

```
OLLAMA_HOST=127.0.0.1:11435
```

08 · VALIDATION

Seven scenarios, all verified

Scenario	What it simulates	Result
01 Benign OpenAI use	Normal coding questions	0 findings, 0 incidents
02 Shadow AI traffic	python.exe → Ollama, node.exe → HF	SHADOW in inventory only
03 Credential leak	8 key types to approved and shadow AI	Caught: 4 Medium / 6 High
04 PII leak	Email, +91 phone, PAN, Aadhaar, card	Caught: 3+ items → +3
05 Agent chain	.env → shell → AWS key to OpenAI	1 incident · 10/10 Critical
06 False-positive bait	UUIDs, hashes, images, bad numbers	0 findings
07 Agent variety	Claude agent + ~/.ssh key; curl | sh	Critical; Medium

81 automated checks pass · 0 raw secrets in the database dump · server bound to 127.0.0.1 only

08 · REQUIREMENTS

Every requirement, implemented

Problem statement asks	How ShadowGuard AI does it
1 · Inspect connections or log uploads	psutil collector (metadata) + JSONL request-log ingest + Ollama relay
2 · Identify AI providers, local LLMs	Host, port (11434, 1234) and API-path registry → approved or SHADOW
3 · Detect secrets & PII (regex + entropy)	8 credential rules, 5 PII rules, Luhn / Verhoeff / JWT validators, entropy
4 · Flag risky agent behaviour	Shell and file events + tool_calls / tool_use parsing + 60 s chain
5 · Local, privacy-first dashboard	React dashboard on 127.0.0.1: stats, alerts, explainable 0–10 scores
Constraint · no SSL decryption	Metadata for HTTPS; payloads only where already plaintext
Sponsor · Featherless	Redacted-only AI triage with summary and remediation; offline fallback

09 · DEPLOYMENT & ROADMAP

From one laptop to a whole company

For teams without enterprise DLP — and where DLP is blind: local LLMs and agents

HOW A COMPANY ROLLS IT OUT

1. Package the agent as an MSI / pkg service
2. Push it to company laptops via Intune or Jamf
3. It detects locally on each device
4. Sends only metadata + redacted findings over HTTPS
5. Security team sees every device in one dashboard

No install option: firewall, DNS or VPN logs feed the same ingest API.

ROADMAP

Browser extension (prompts before encryption) · AI-gateway log connector · central multi-device server · optional enforcement

HONEST LIMITS TODAY

Encrypted HTTPS: metadata only · detects, does not block · rule-based patterns · single-machine prototype

TEAM CONVERGE · TEAM 48

ShadowGuard AI

See every AI tool. Catch every leaked secret. Never read what you shouldn’t.

---

github.com/Muzammil757/HFT26-TEAM-48

Thank you · Questions?